

Title: Augmented Bilinear Network for Incremental Multi-Stock Time-Series Classification

Type of article: Article

Authors:

S No.	Name(s) of Author(s)	Highest academic degree	Author Affiliations	Email address
1	Mostafa Shabani	M.Sc.	Department of Electrical and Computer Engineering, Aarhus University, Denmark	mshabani@ece.au.dk
2	Dat Thanh Tran	M.Sc.	Department of Computing Sciences, Tampere University, Finland	thanh.tran@tuni.fi
3	Juho Kanninen	PhD	Department of Computing Sciences, Tampere University, Finland	juho.kanninen@tuni.fi
4	Alexandros Iosifidis	PhD	Department of Electrical and Computer Engineering, Aarhus University, Denmark	ai@ece.au.dk

Corresponding author: Mostafa Shabani

Number of References: 62

Source of support:

The research received funding from the Independent Research Fund Denmark project DISPA (Project Number: 9041-00004).

Highlights

Augmented Bilinear Network for Incremental Multi-Stock Time-Series Classification

Mostafa Shabani, Dat Thanh Tran, Juho Kanninen, Alexandros Iosifidis

- We describe a new research problem that is motivated by situations frequently arising in financial time-series analysis problems;
- We propose a solution to the research problem based on the TABL network, the effectiveness of which is demonstrated through experiments on two frequently occurring scenarios;
- We show that the proposed approach can be extended to other neural network types by providing a solution based on CNN;
- We improve the time and space complexities of the optimization process of our solution by using low rank tensor representations.

Augmented Bilinear Network for Incremental Multi-Stock Time-Series Classification

Mostafa Shabani^{a,*}, Dat Thanh Tran^b, Juho Kanninen^b and Alexandros Iosifidis^a

^a*DIGIT, Department of Electrical and Computer Engineering, Aarhus University, Denmark*

^b*Unit of Computing Sciences, Tampere University Tampere, Finland*

ARTICLE INFO

Keywords:

Deep Learning
Low Rank tensor decomposition
Limit Order Book data
Financial time-series analysis

ABSTRACT

Deep Learning models have become dominant in tackling financial time-series analysis problems, overturning conventional machine learning and statistical methods. Most often, a model trained for one market or security cannot be directly applied to another market or security due to differences inherent in the market conditions. In addition, as the market evolves over time, it is necessary to update the existing models or train new ones when new data is made available. This scenario, which is inherent in most financial forecasting applications, naturally raises the following research question: *How to efficiently adapt a pre-trained model to a new set of data while retaining performance on the old data, especially when the old data is not accessible?* In this paper, we propose a method to efficiently retain the knowledge available in a neural network pre-trained on a set of securities and adapt it to achieve high performance in new ones. In our method, the prior knowledge encoded in a pre-trained neural network is maintained by keeping existing connections fixed, and this knowledge is adjusted for the new securities by a set of augmented connections, which are optimized using the new data. The auxiliary connections are constrained to be of low rank. This not only allows us to rapidly optimize for the new task but also reduces the storage and run-time complexity during the deployment phase. The efficiency of our approach is empirically validated in the stock mid-price movement prediction problem using a large-scale limit order book dataset. Experimental results show that our approach enhances prediction performance as well as reduces the overall number of network parameters.

1. Introduction

Deep Learning has become prominent in tackling challenges in financial time series analysis [1, 2, 3, 4, 5, 6, 7, 8, 9]. Since the inception of electronic trading systems, large amounts of trade data have become available and accessible to many people. With large-scale data, conventional approaches based on linear and non-linear models trained with convex optimization have become less efficient in terms of computational complexity as well as prediction performance. Significant improvements in computing hardware coupled with increasing amounts of large-scale datasets have enabled the research community to harness the power of deep neural networks combined with stochastic optimization. The shift towards an end-to-end Deep Learning paradigm not only allows us to tackle larger and more complex problems but also enables us to focus on more important aspects of the real-world problems, such as the feasibility and efficiency when applying learning models to real-world financial problems [10].

Although it is less demanding in terms of memory complexity compared to non-linear models trained with convex optimization, optimizing deep neural networks using stochastic optimization is still time-consuming, since in order to train a well-performing network often requires a high number of iterations of mini-batch based optimization. The high computational cost is also associated with high energy usage, which can significantly affect the profitability of a Deep Learning solution. On the microeconomic level, since the data throughput is large and the frequency of changes in the data distribution is high, especially in highly liquid markets, we are faced with a continuous influx of data. On the macroeconomic level, the market continuously evolves to accommodate different phases of the economy. These unique features of the financial market can easily make a data-driven system obsolete, requiring it to be frequently updated with recent data. The necessity of frequent updates coupled with the high cost of updates has made computational complexity a critical factor when utilizing deep neural networks in financial analysis or forecasting systems.

*Corresponding author

✉ mshabani@ece.au.dk (M. Shabani); thanh.tran@tuni.fi (D.T. Tran); juho.kanninen@tuni.fi (J. Kanninen); ai@ece.au.dk (A. Iosifidis)

There have been different approaches to reduce computational complexity when training deep neural networks, such as designing novel low-complexity network architectures [11, 12, 13, 14], replacing existing ones with their low-rank counterparts [15, 16, 17, 18], or adapting the pre-trained models to new tasks, i.e., performing Transfer Learning (TL) [19, 20, 21] or Domain Adaptation (DA) learning [22, 23, 24, 25]. Among these approaches, model adaptation is the most versatile since a method in this category is often architecture-agnostic, being complementary to other approaches. In financial analysis or forecast settings, the ability to reuse existing pre-trained models can play an important role. The need to update a system on a regular basis arises in many situations in finance [26, 27, 28]. Instead of training a new model from scratch with the new data, an efficient adaptation method will allow us to quickly adjust the existing model using the new data with less computational burden. Not only can this approach reduce the operating costs, thereby increasing the profit of the system, but also create the flexibility of more frequent updates. Besides the computational efficiency, model re-usability might also improve the modeling performance of a system. For example, data might be abundant for highly liquid markets/securities but might be scarce for less liquid markets/securities. As a result, we might not have sufficient data of an illiquid stock to train a deep neural network from scratch without overfitting. By taking advantage of models pre-trained on other stocks, one can obtain better performance even with a small amount of data in new stocks.

In this paper, we consider the following research problem: given a new task $\mathcal{T}_{\text{new}}$ defined on financial time-series data and a neural network $\mathcal{N}_{\text{old}}$, which has been trained on previously collected financial time-series data to solve a task $\mathcal{T}_{\text{old}}$ that is relevant to $\mathcal{T}_{\text{new}}$, the objective is to efficiently generate a new network $\mathcal{N}_{\text{new}}$ based on $\mathcal{N}_{\text{old}}$ that generalizes well for the new task $\mathcal{T}_{\text{new}}$ without having access to the data defining $\mathcal{T}_{\text{old}}$, and without harming the performance of $\mathcal{N}_{\text{old}}$ in $\mathcal{T}_{\text{old}}$. The tasks are often expressed via their datasets, and we refer to the datasets of $\mathcal{T}_{\text{old}}$ and $\mathcal{T}_{\text{new}}$ as $\mathcal{D}_{\text{old}}$ and $\mathcal{D}_{\text{new}}$, respectively. We refer to the above-described problem as the *Incremental Multi-Stock Time-Series Analysis* problem. While multiple time-series analysis tasks can be formulated in this context, we focus on the case where the $\mathcal{T}_{\text{old}}$ corresponds to a mid-price direction prediction task defined on a set of stocks. This task can be formulated as a time-series classification problem. Historic data of $\mathcal{T}_{\text{old}}$ forming $\mathcal{D}_{\text{old}}$ is used to train the Deep Learning model $\mathcal{N}_{\text{old}}$. Given a pre-trained $\mathcal{N}_{\text{old}}$ and historic data of a (set of) new stock(s) forming $\mathcal{D}_{\text{new}}$, we would like to exploit the knowledge encoded in $\mathcal{N}_{\text{old}}$ to effectively be able to predict the direction of mid-price movements of the stocks belonging to both $\mathcal{T}_{\text{old}}$ and the stock(s) defining the new task $\mathcal{T}_{\text{new}}$.

The research problem described above is highly relevant to stock market data analysis since one gets access to historic data of new stocks in different time periods, and historic data used to train existing models can be either absent or so big that merging data in $\mathcal{D}_{\text{old}}$ and $\mathcal{D}_{\text{new}}$ to train $\mathcal{D}_{\text{new}}$ can be impractical due to their large size. While TL and DA have been widely adopted to solve related problems, they are not well-suited for addressing the problem of interest in our study. This is due to that they either require the use of both $\mathcal{D}_{\text{old}}$ and $\mathcal{D}_{\text{new}}$ to *adapt* the parameters of $\mathcal{N}_{\text{old}}$ to the new task, or they create a second model $\mathcal{N}_{\text{new}}$ the parameters of which are initialized to those of $\mathcal{N}_{\text{old}}$ and are further fine-tuned using $\mathcal{D}_{\text{new}}$, thus leading to an inefficient solution.

In this paper, we propose a method for performing network augmentation by learning auxiliary neural connections that are complementary to the existing ones. This allows the new model to preserve prior knowledge and rapidly adapt to the new tasks, leading to improvements in performance. By using a low-rank approximation for the auxiliary connections, our method obtains additional efficiency in terms of overall operational cost. We demonstrate our approach with the Temporal Attention-augmented Bilinear Layer (TABL) [29] network architecture, which achieves state-of-the-art performance in the stock mid-price direction prediction. In addition, we also demonstrate that the proposed approach can generalize to Convolutional Neural Networks (CNN), which is another type of neural network architecture that is widely used in financial time series analysis [2, 1]. Using a large-scale Limit Order Book (LOB) dataset, our empirical study shows that the proposed method can indeed improve both prediction performance and operational efficiency of TABL networks as well as CNN networks, compared to TL and DA approaches.

The remainder of the paper is organized as follows. Related works in financial time series analysis are briefly presented in Section 2. The proposed method is described in Section 3. The complexity analysis of the proposed method is provided in section 3.3. In Section 4, we describe in detail our experimental protocol and present the empirical results. Section 5 concludes our work and discusses potential future research directions.

2. Related work

While econometric models can provide certain statistical insights with great transparency [30, 31], deep neural networks following the end-to-end data-driven learning paradigm led to significant improvements on the performance

of financial time series prediction tasks. In [1], a methodology based on Convolutional Neural Networks (CNN) was proposed for mid-price direction prediction. Deep Learning models utilizing CNN to capture the spatial structure of the LOB and Long Short Term Memory (LSTM) units to capture time dependencies were proposed in [2, 32]. A multilayer Perceptron was used in [5] in the form of a spatial neural network. Adaptive input normalization jointly optimizing the input data normalization in the form of a neural layer with the parameters of the Deep Learning used for classification was proposed in [4, 33].

2.1. Model Re-usability

To address model re-usability, TL enables exploiting information from a pre-trained model that was trained on a source dataset, referred to as the base model hereafter, to improve the performance of the target model being trained on a target dataset [34]. Domains of the source and target datasets can be different, however, they must be related to each other. In cases where the source and target domains are highly dissimilar, negative transfer issues can appear, i.e., transferring knowledge adversely affects the performance of the target model [35]. In [36], the Dynamic Time Warping method (DTW) [37] was used to find similarities between the source and target datasets to avoid the negative transfer issue in TL of deep CNNs for time series classification tasks. Autonomous transfer learning (ATL) was proposed in [38] to produce a domain invariant network and handle the problem of concept drifts [39] both in the source and target domains. A hybrid algorithm based on TL, i.e. the Online Sequential Extreme Learning Machine with Kernels (OS-ELMK), and ensemble learning for time series prediction was proposed in [40] to handle the problem of wide variability between old data (base dataset) and new data (target dataset). In [41], a fuzzy regression TL method was proposed, using the Takagi-Sugeno fuzzy regression model to transfer knowledge from a source domain to a target domain. Data augmentation was used to improve the performance of TL for stock price direction prediction in [21]. A TL framework for predicting stock price movements that uses relationships between stocks to construct effective input was proposed in [42]. In [43], a TL architecture using encoder-decoder was proposed to learn market-specific trading strategies. Online transfer learning [44, 45] tries to handle the transfer learning problem within an online learning process. In these types of problems, we use the knowledge from some source domains to improve the performance of an online learning task in the target domain. Incremental Learning (IL) [46] is another approach that can be used when we are dealing with a data stream and the prediction performance is reduced due to changes in the feature space of the new task. These approaches try to update some characteristics of fixed network structure to handle the problem of the new task. In [47], neural networks with dynamically evolved capacity (NADINE) are proposed, which can update the network structure and improve the prediction performance based on changes in the learning environment.

DA is another model re-usability approach. DA methods are transductive TL approaches with the assumption that the distributions of the source dataset and target dataset are different [48]. In [49], a feature learning approach was proposed that provides domain-invariant features. In [50], a Deep Adaptation Network (DAN) architecture was proposed that uses maximum mean discrepancy (MMD) [51] to find a domain-invariant feature space. In [24], it was shown that supervised DA can be seen as a two-view Graph Embedding. In [52], a method was proposed that combines DA techniques and drift handling mechanism to solve the multistream classification problem under multisource streams. In multistream classification [52, 53], we have two datasets of source and target stream data which come from the same domain. The source stream dataset consists of labeled data, while the target stream dataset is unlabeled. The task is to predict the class labels of the target stream data and address challenges related to infinite length and concept drift. In this type of problem, we have access to the data of both the source and target data. One of the major differences between TL and DA is that DA requires all the data of both the source and target domains. As the source and target models need to be trained jointly, it is memory-intensive since the parameters of both models need to be updated [34]. When there is a major difference between the distribution of the base and target datasets, the performance of the base model may deteriorate. Multi-domain learning [54] methods incorporate the properties of multi-task learning [55] and domain adaptation. In multi-domain learning, the goal is to handle the same problem for different domains. In [56], an adaptive method for multi-domain learning is proposed that reduced the required base model parameters based on the complexity of the different domains coming from image classification problems.

2.2. Temporal Attention-augmented Bilinear Layer

We will demonstrate our model augmentation approach with an instantiation of the Temporal Attention-augmented Bilinear Layer (TABL) network [29], which has been proposed as an efficient and effective neural network architecture for financial time-series classification. In the following, we describe the working mechanism of the TABL network architecture, providing necessary details to understand our method described in Section 3.

TABL combines the ideas of bilinear projection and attention mechanism. The input to a TABL is a multivariate time-series $\mathbf{X} \in \mathbb{R}^{D \times T}$, with T denoting the number of instances combined in the temporal dimension to form the time-series and D denoting the dimensionality of the instances forming the time-series. Given an input to the TABL, it generates an output multivariate time-series $\mathbf{Y} \in \mathbb{R}^{D' \times T'}$, where T' and D' denote the transformed number of instances and their dimensionality, respectively. This is performed through five computation steps:

1. A feature transformation of the input $\mathbf{X}$ is performed using a weight matrix $\mathbf{W}_1 \in \mathbb{R}^{D' \times D}$, producing the intermediate feature matrix $\tilde{\mathbf{X}} \in \mathbb{R}^{D' \times T}$:

$$\tilde{\mathbf{X}} = \mathbf{W}_1 \mathbf{X}. \quad (1)$$

2. The relative importance of the instances forming the time-series is computed by:

$$\mathbf{E} = \tilde{\mathbf{X}} \mathbf{W}, \quad (2)$$

where $\mathbf{W} \in \mathbb{R}^{T \times T}$ is a structured matrix that has fixed diagonal elements equal to $1/T$. By learning non-diagonal elements, the matrix $\mathbf{W}$ expresses weights encoding the pair-wise instance importance in the transformed feature space $\mathbb{R}^{D'}$, while the self-importance of all instances is set to be equal.

3. The importance scores stored at the elements e_{ij} of $\mathbf{E}$ are normalized in a row-wise manner to produce an attention mask $\mathbf{A} \in \mathbb{R}^{D' \times T}$ formed by the attention scores. This is done using the soft-max function:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^T \exp(e_{ik})}. \quad (3)$$

Each row of the attention matrix $\mathbf{A}$ sums up to 1, distributing the importance scores along the time dimension for each dimension of the transformed input time-series data.

4. The attended features $\tilde{\tilde{\mathbf{X}}} \in \mathbb{R}^{D' \times T}$ are computed by applying the attention mask to $\tilde{\mathbf{X}}$. To enable a soft attention, a learnable parameter λ is used to weight the contribution of the attended $\tilde{\mathbf{X}} \odot \mathbf{A}$ and the original features $\tilde{\mathbf{X}}$:

$$\tilde{\tilde{\mathbf{X}}} = \lambda(\tilde{\mathbf{X}} \odot \mathbf{A}) + (1 - \lambda)\tilde{\mathbf{X}}. \quad (4)$$

λ is constrained to have a value between $[0, 1]$.

5. The final attended features $\tilde{\tilde{\mathbf{X}}}$ are linearly transformed in the second tensor mode by weight matrix $\mathbf{W}_2 \in \mathbb{R}^{T \times T'}$, shifted by the bias $\mathbf{B} \in \mathbb{R}^{D' \times T'}$, and activated by a nonlinear element-wise activation function $\phi(\cdot)$, e.g., the Rectified Linear Unit (ReLU) function:

$$\mathbf{Y} = \phi(\tilde{\tilde{\mathbf{X}}} \mathbf{W}_2 + \mathbf{B}). \quad (5)$$

A schematic illustration of the above computation steps is provided in Figure 1.

A TABL network is created by combining multiple TABLs or by combining TABLs with Bilinear Layers (BL). A BL is a layer in which the temporal attention branch is not used, or equivalently the value of parameter λ is set to zero.

3. Proposed Method

The proposed method is based on two main ideas, i.e. model augmentation and low-rank approximation. Although the method presented in this Section is described in detail for the TABL network architecture, this approach can be easily generalized to other architectures, for example, the Convolutional Neural Network architecture as we will show later in this section.

Based on the problem definition described in the Introduction, we want to use the information encoded in a $\mathcal{N}_{\text{old}}$ that was trained on $\mathcal{D}_{\text{old}}$ to improve the prediction performance for $\mathcal{D}_{\text{new}}$ with the following restrictions:

1. without using the data in $\mathcal{D}_{\text{old}}$, as this data may not be available or may be very big leading to very computationally costly training for $\mathcal{D}_{\text{new}}$,
2. without harming the performance of $\mathcal{N}_{\text{old}}$ on the original task, i.e., $\mathcal{T}_{\text{old}}$,
3. without increasing the memory and the computation requirements much for operating on both tasks $\mathcal{T}_{\text{old}}$ and $\mathcal{T}_{\text{new}}$, after $\mathcal{N}_{\text{new}}$ is trained and deployed.

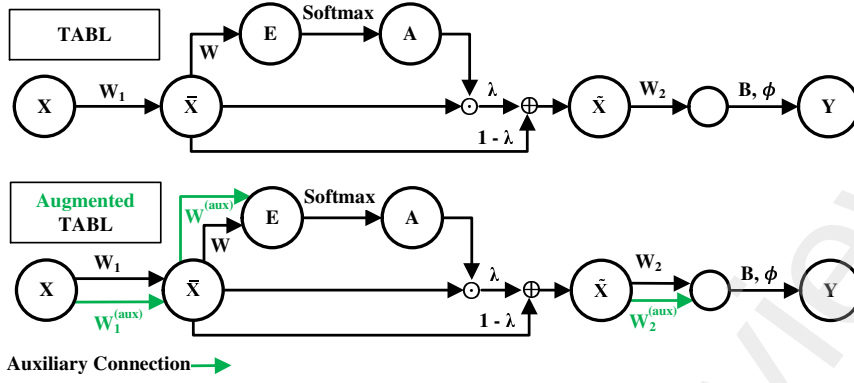


Figure 1: Illustration of TABL (top) and the proposed Augmented TABL (bottom). Auxiliary connections are shown in green color.

This is achieved by augmenting the parameters $\Theta_{\mathcal{N}_{old}}$ of $\mathcal{N}_{old}$ with auxiliary parameters Θ_{aux} . That is, $\mathcal{N}_{new}$ is constructed by adding additional connections to the pre-trained neural network $\mathcal{N}_{old}$. In order to retain the prior knowledge in $\mathcal{N}_{old}$, its parameters $\Theta_{\mathcal{N}_{old}}$ are fixed and we only optimize the auxiliary parameters Θ_{aux} to learn additional information needed to perform well on the stocks defining $\mathcal{T}_{new}$.

There are different strategies to incorporate auxiliary connections to a pre-trained model. One of the most common approaches in transfer learning is to add new hidden layers after the last hidden layer, thereby extending the network's depth and replacing the prediction layer, also known as the penultimate layer. Instead of modifying the pre-trained model's topology, i.e., the number of layers and the size of each layer, we propose to augment the pre-trained model with auxiliary connections that are parallel to the existing ones, thereby keeping the original architecture design unchanged. The intuition behind this approach is that the architectural choice of a pre-trained model is often validated and obtained by extensive experimentation process. It has been shown that many state-of-the-art network architectures such as ResNet-50 [57] or DenseNet121 [58] are not specific to a dataset, but perform well in many similar problems. Thus, by respecting the architectural choices of the pre-trained network $\mathcal{N}_{old}$, we can avoid the time-consuming process of validating architectural choices for the auxiliary connections.

3.1. Augmented TABL

The proposed model augmentation method is illustrated in Figure 1. As can be seen from this figure, we incorporate into the pre-trained TABL three auxiliary connections (depicted in green color), which are parameterized by $\mathbf{W}_1^{(aux)}$, $\mathbf{W}^{(aux)}$ and $\mathbf{W}_2^{(aux)}$, respectively. The dimensions of auxiliary parameters are exactly the same as those that they are augmenting. Particularly, $\mathbf{W}_1^{(aux)} \in \mathbb{R}^{D' \times D}$ is added to augment $\mathbf{W}_1$; $\mathbf{W}^{(aux)} \in \mathbb{R}^{T \times T}$ is added to augment $\mathbf{W}$; and $\mathbf{W}_2^{(aux)} \in \mathbb{R}^{T \times T'}$ is added to augment $\mathbf{W}_2$. The transformations produced by the augmented TABL are described by the following equations:

$$\tilde{\mathbf{X}} = \mathbf{W}_1 \mathbf{X} + \mathbf{W}_1^{(aux)} \mathbf{X}, \quad (6)$$

$$\mathbf{E} = \tilde{\mathbf{X}} \mathbf{W} + \tilde{\mathbf{X}} \mathbf{W}^{(aux)}, \quad (7)$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^T \exp(e_{ik})}, \quad (8)$$

$$\tilde{\tilde{\mathbf{X}}} = \lambda(\tilde{\mathbf{X}} \odot \mathbf{A}) + (1 - \lambda)\tilde{\mathbf{X}}, \quad (9)$$

$$\mathbf{Y} = \phi(\tilde{\tilde{\mathbf{X}}} \mathbf{W}_2 + \tilde{\tilde{\mathbf{X}}} \mathbf{W}_2^{(aux)} + \mathbf{B}). \quad (10)$$

In Eq. (6), the intermediate feature matrix that is formed using the $\mathbf{W}_1$ of pre-trained model is added to the transformed features matrix that is produced using auxiliary parameter $\mathbf{W}_1^{(aux)}$. The matrix $\mathbf{E}$ (Eq. (7)) which shows the relative importance of instances is produced by summing up the output of Eq. (6) with both $\mathbf{W}$ and $\mathbf{W}^{(aux)}$. In the Eq. (10),

the final attended features that are produced in Eq. (9) are multiplied by both $\mathbf{W}_2$ and $\mathbf{W}_2^{(\text{aux})}$ and added together. The output of this summation is shifted by $\mathbf{B}$ and activated by an activation function. As can be seen from Eqs. (6)-(10), for each computation step that involves a weight matrix, we retain the transformations used by the pre-trained TABL, while learning additional (complementary) transformations through the auxiliary parameters ($\mathbf{W}_1^{(\text{aux})}$, $\mathbf{W}^{(\text{aux})}$, $\mathbf{W}_2^{(\text{aux})}$). By making the auxiliary computation steps mimic those in the original TABL, we respect the architectural design of the pre-trained model. In addition, since only the auxiliary parameters are updated during the optimization process, intermediate knowledge of the pre-trained model (preserved in its weight matrices) is still retained and we only learn auxiliary information to improve performance in the new task at hand.

3.2. Low Rank Augmented TABL

Besides model augmentation, our method exploits the idea of low-rank approximation. That is, we enforce a constraint (an upper-bound K) to the rank of each auxiliary weight matrix by representing it as a two-factor decomposition. Specifically, we define the auxiliary weight matrices as:

$$\mathbf{W}_1^{(\text{aux})} = \mathbf{L}_1 \mathbf{R}_1, \quad (11)$$

$$\mathbf{W}^{(\text{aux})} = \mathbf{L} \mathbf{R}, \quad (12)$$

$$\mathbf{W}_2^{(\text{aux})} = \mathbf{L}_2 \mathbf{R}_2, \quad (13)$$

where $\mathbf{L}_1 \in \mathbb{R}^{D' \times K}$, $\mathbf{R}_1 \in \mathbb{R}^{K \times D}$, $\mathbf{L} \in \mathbb{R}^{T \times K}$, $\mathbf{R} \in \mathbb{R}^{K \times T}$, $\mathbf{L}_2 \in \mathbb{R}^{T \times K}$, $\mathbf{R}_2 \in \mathbb{R}^{K \times T'}$, with $K \leq \min(D, D', T, T')$ being a hyperparameter value.

By constraining the value of K to a small number, we enforce the weight matrices $\mathbf{W}_1^{(\text{aux})}$, $\mathbf{W}^{(\text{aux})}$ and $\mathbf{W}_2^{(\text{aux})}$ to have low ranks. The advantage of the low-rank approximation is two-fold. The first advantage is the reduction in computational and memory complexities during the optimization process. This will be analyzed in detail in the next subsection. In addition to improvements in complexity, the use of low-rank approximation can also have a regularization effect, thus improving the learning performance of the neural network. This is because low-rank approximation reduces the degrees of freedom in a given transformation, i.e., the number of parameters to be estimated, thus, it potentially reduces the overfitting effect when sufficient training data is not available.

To summarize, to take advantage of the pre-trained model to solve the new task $\mathcal{T}_{\text{new}}$ defined by the dataset $\mathcal{D}_{\text{new}}$, we solve the following optimization objective:

$$\arg \min_{\substack{\mathbf{L}_1^{(l)}, \mathbf{L}^{(l)}, \mathbf{L}_2^{(l)}, \\ \mathbf{R}_1^{(l)}, \mathbf{R}^{(l)}, \mathbf{R}_2^{(l)} \\ l=1, \dots, L}} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(\mathcal{N}_{\text{new}}(x_i, y_i)), \quad (14)$$

where $(x_i, y_i) \in \mathcal{D}_{\text{new}}$ denotes the i -th training time-series and the corresponding target in the new dataset, L denotes the number of TABLs in the new model, and $\mathcal{L}$ denotes the loss function.

3.3. Complexity Analysis

In this subsection, we provide our analysis on the complexity of the proposed method, as well as some notes on the implementation details. The first thing we should point out about the complexity of our method is that the inference complexity of the new model $\mathcal{N}_{\text{new}}$ is exactly the same as that of the pre-trained model $\mathcal{N}_{\text{old}}$. In other words, after the transfer learning process, the new model induces the same operating cost as the old model. This is because of the distributive property of matrix multiplications in Eq. (6), (7), (10). Let us denote:

$$\begin{aligned} \mathbf{W}_1^{(\text{new})} &= \mathbf{W}_1 + \mathbf{W}_1^{(\text{aux})}, \\ \mathbf{W}^{(\text{new})} &= \mathbf{W} + \mathbf{W}^{(\text{aux})}, \\ \mathbf{W}_2^{(\text{new})} &= \mathbf{W}_2 + \mathbf{W}_2^{(\text{aux})}. \end{aligned} \quad (15)$$

Eqs. (6), (7) and (10) become:

$$\tilde{\mathbf{X}} = \mathbf{W}_1^{(\text{new})} \mathbf{X}, \quad (16)$$

$$\mathbf{E} = \tilde{\mathbf{X}} \mathbf{W}^{(\text{new})}, \quad (17)$$

$$\mathbf{Y} = \phi \left(\tilde{\mathbf{X}} \mathbf{W}_2^{(\text{new})} + \mathbf{B} \right). \quad (18)$$

After training, we can simply compute Eq. (15) once and only store the values of $\mathbf{W}_1^{(\text{new})}$, $\mathbf{W}^{(\text{new})}$, $\mathbf{W}_2^{(\text{new})}$ for inference. In this way, we do not need to retain both the values of $\mathbf{W}_1, \mathbf{W}, \mathbf{W}_2$ of the pre-trained model and the values of $\mathbf{L}_1, \mathbf{R}_1, \mathbf{L}, \mathbf{R}, \mathbf{L}_2, \mathbf{R}_2$ of the auxiliary connections, which may consume more storage space than $\mathbf{W}_1^{(\text{new})}, \mathbf{W}^{(\text{new})}, \mathbf{W}_2^{(\text{new})}$. Since the matrix dimensions in Eqs. (16)-(18) for the new model are exactly the same as those in Eqs. (1), (2) and (5) for the pre-trained model, our method does not introduce any additional complexity during the inference/deployment phase.

The complexity during the training phase is dependent on the implementation details. Here we should note that there are two strategies to implement the proposed augmented TABL:

- **Implementation Strategy 1 (IS1):** during the forward pass, using Eqs. (11)-(13), we first compute $\mathbf{W}_1^{(\text{aux})}$, $\mathbf{W}^{(\text{aux})}$ and $\mathbf{W}_2^{(\text{aux})}$ from $\mathbf{L}_1, \mathbf{R}_1, \mathbf{L}, \mathbf{R}, \mathbf{L}_2$ and $\mathbf{R}_2$. After that, using Eq. (15), we can compute $\mathbf{W}_1^{(\text{new})}$, $\mathbf{W}^{(\text{new})}$, $\mathbf{W}_2^{(\text{new})}$, which are then used in Eqs. (16)-(18) to compute the output of the augmented TABL.
- **Implementation Strategy 2 (IS2):** in this case we do not compute explicitly the auxiliary weight matrices $\mathbf{W}_1^{(\text{aux})}$, $\mathbf{W}^{(\text{aux})}$ and $\mathbf{W}_2^{(\text{aux})}$ but we take advantage of the size of their low-rank approximations. Specifically, Eqs. (6), (7) and (10) are computed as follows, with the computation order from left to right and by respecting the priority of the parentheses:

$$\tilde{\mathbf{X}} = \mathbf{W}_1 \mathbf{X} + \mathbf{L}_1 (\mathbf{R}_1 \mathbf{X}), \quad (19)$$

$$\mathbf{E} = \tilde{\mathbf{X}} \mathbf{W} + (\tilde{\mathbf{X}} \mathbf{L}) \mathbf{R}, \quad (20)$$

$$\mathbf{Y} = \phi \left(\tilde{\mathbf{X}} \mathbf{W}_2 + (\tilde{\mathbf{X}} \mathbf{L}_2) \mathbf{R}_2 + \mathbf{B} \right). \quad (21)$$

Each of the above-mentioned strategies has its own advantages in terms of computational and memory complexity. Adopting the first implementation strategy (IS1) will lead to a faster forward-backward pass compared to the second strategy (IS2). This is because during stochastic optimization, we often update multiple samples in the same forward-backward pass, thus leading to $\mathbf{X}$ having another large dimension, which corresponds to the mini-batch size. Since each equation in IS2 involves two matrix multiplications with $\mathbf{X}$ or $\tilde{\mathbf{X}}$ or $\tilde{\mathbf{X}}$, IS2 requires more calculations compared to IS1. The computational complexity estimates of IS1 and IS2 can be found in the Appendix.

While adopting IS1 can lead to a faster forward-backward pass compared to IS2, this strategy also requires a higher amount of memory, which might not be feasible when training large networks on a GPU with limited memory. This is because using IS2, during the forward pass, we do not need to keep the intermediate outputs of $\mathbf{W}_1 \mathbf{X}$ in Eq. (19), $\tilde{\mathbf{X}} \mathbf{W}$ in Eq. (20) and $\tilde{\mathbf{X}} \mathbf{W}_2$ in Eq. (21) for the gradient update computation in the backward pass. On the other hand, when using IS1 we need to keep all intermediate outputs in the forward pass in order to compute the gradient updates for the backward pass. In addition, since the auxiliary weight matrices are explicitly computed in the forward pass of IS1, we do not obtain any memory reduction from using the low-rank approximation as is the case for IS2.

3.4. Augmented Convolution Layer

As we mentioned in the beginning of this section, the proposed approach can be easily generalized to other architectures since most neural networks rely on linear or multilinear transformations. For Convolutional Neural Networks (CNNs), an augmented convolution layer can be formed by incorporating low-rank auxiliary filters to the pre-trained filters in a convolution layer. Since the filters in a convolution layer can be represented as a 3-mode (1D convolution layer) or 4-mode (2D convolution layer) tensor, the low-rank auxiliary filters can be represented in the Canonical Polyadic (CP) form [59], similar to the low-rank CNN proposed in [16]. Let us denote $\mathcal{W}$ as the pre-trained convolution filters in a convolution layer, and $\mathbf{X}$ as the input time-series. Similar to an augmented TABL,

The augmented convolution layer has the following form:

$$\mathbf{Y} = \mathbf{X} \circledast (\mathcal{W} + \mathcal{W}^{(\text{aux})}) \quad (22)$$

$$= \mathbf{X} \circledast \mathcal{W} + \mathbf{X} \circledast \mathcal{W}^{(\text{aux})}, \quad (23)$$

where $\mathbf{Y}$ denotes the output of the convolution layer, $\circledast$ denotes the convolution operation, and $\mathcal{W}^{(\text{aux})}$ denotes the low-rank auxiliary filters.

The convolutional architectures for time-series often consist of 1D convolution layers. Thus, $\mathcal{W}$ is a 3-mode tensor, i.e., $\mathcal{W} \in \mathbb{R}^{N \times D \times t}$ with N denotes the number of filters, D denotes the number of time-series and t denotes the kernel size. Using the CP form, $\mathcal{W}^{(\text{aux})}$ can be represented as:

$$\mathcal{W}^{(\text{aux})} = \sum_{k=1}^K \mathbf{w}_1(k) \otimes \mathbf{w}_2(k) \otimes \mathbf{w}_3(k), \quad (24)$$

where $\mathbf{w}_1(k) \in \mathbb{R}^{N \times 1 \times 1}$, $\mathbf{w}_2(k) \in \mathbb{R}^{1 \times D \times 1}$, $\mathbf{w}_3(k) \in \mathbb{R}^{1 \times 1 \times t}$. $\otimes$ denotes the outer product and K denotes the rank hyperparameter.

4. Experiments

4.1. Dataset and Experimental Protocols

In order to evaluate the effectiveness of the proposed method in the research problem defined in Section 1, we conducted experiments in the problem of stock mid-price direction prediction using the information appearing in Limit Order Books (LOB). Mid-price is the average value between the best bid (buy) price and the best ask (sell) price of a given stock. Although this quantity is just a virtual quantity, i.e., at a given time instance, no transaction can happen at the mid-price, the direction of mid-price change can capture the dynamics of a given stock, reflecting the supply and demand in the market. For this reason, mid-price direction prediction is a popular problem when analyzing LOB information [60].

The dataset used in our experiments is a public LOB dataset known as FI-2010 [61], which consists of more than four million limit orders during the period of ten working days (from 1st of June to 14th of June 2010). The dataset contains limit orders for five companies traded in the Helsinki Exchange, operated by Nasdaq Nordic. At each time instance, the dataset provides the quotes of the top ten levels of the LOB, i.e., the top ten best bid prices and volumes and top ten best ask prices and volumes. The top quotes form 40 different values corresponding to the bid and ask prices and volumes of the top 10 LOB levels. Regarding the labels of the mid-price, at any given time instance, the database provides the direction (stationary, increasing, decreasing) of the mid-price after the next H time instances, where $H \in \{10, 20, 30, 50, 100\}$ denotes the prediction horizon.

We followed a similar experimental protocol as in [29]: the input to all of the evaluated models was formed from the 10 most recent limit order events, which consists of 10 instances of 40 dimensions standardized using z-score normalization. As has been shown in [62], the adoption of information from all 10 LOB levels is important for achieving high performance in learning-based methods. All models were trained to predict the mid-price direction after 10 events, i.e., $H = 10$. Regarding the train and test datasets, we used time-series of the first seven days for training and time-series of the last three days for testing. From the training set, we used the last 10% of the time-series samples for validation purposes. In addition, we also evaluated our model augmentation strategy in an online learning setting, which is described in detail in Section 4.4. All models were optimized using the Adam optimizer with an initial learning rate of 0.01. The learning rate was reduced whenever the validation loss reached a plateau.

Since FI-2010 is an imbalanced dataset with the majority of labels belonging to the stationary class, F1-score is used as the main performance metric, similar to prior works [29, 2, 4]. The class distributions of 5 stocks individually and the overall class distributions for the dataset can be seen in Table 1. In addition to F1-score, we also report average accuracy, precision and recall. Finally, we adopted the same weighted entropy loss function as in [29] to alleviate the effects of class imbalance:

$$L = - \sum_{c=1}^3 \frac{\beta}{N_c} y_c \log(\tilde{y}_c), \quad (25)$$

Table 1

Distribution of classes in train and test dataset. Class 0 refers to the stock price remains the same, class 1 to the situation where the price goes up, and class 2 where the price goes downs.

Target stock	Train dataset			Test dataset		
	Class 0	Class 1	Class 2	Class 0	Class 1	Class 2
Stock 1	9013	8198	8060	2108	2507	2440
Stock 2	28487	11235	11394	16712	4629	4482
Stock 3	40489	10228	10452	26067	4136	3974
Stock 4	22258	9983	10263	17326	4661	4333
Stock 5	61116	21991	21790	38829	9458	8756
All Stocks	161363 (56%)	61635 (21%)	61959 (21%)	101042 (67%)	25391 (16%)	23985 (15%)

where the N_c is the number of samples in c -th class, and the y_c and $\hat{y}_c$ are the true and predicted probabilities of c -th class respectively. β is a constant with a value of $1e6$. All experiments were run 5 times and we report the mean and standard deviation of 5 runs for each performance metric. Because of the random model parameter initialization and the stochastic nature of the Backpropagation algorithm, it is often the case that we obtain different network parameters, thus different performances at different runs. The use of the standard deviation of the performance achieved over different runs helps us evaluate how stable and reliable our model is given that it is trained using a stochastic optimization process.

4.2. Experimental Setup 1

In order to simulate an experiment following the definition of the research problem defined in Section 1, we conducted experiments for both TABL and CNN architectures with the following setup:

- For each stock S in the database, we consider the data belonging to S as the new dataset D_{new} and the data belonging to the remaining four stocks as the old dataset D_{old} .
- We use the notation TABL-base and CNN-base to denote the models that were trained on the D_{old} dataset. This is the pre-trained model $\mathcal{N}_{\text{old}}$ in our problem formulation. This model, when used directly to evaluate on the new dataset D_{new} , can serve as a simple baseline to compare with our method.
- A TL approach taking advantage of the pre-trained model (TABL-base and CNN-base) and finetuning it on the new dataset D_{new} is also used as a baseline. We denote models following this TL approach as TABL-fine-tune and CNN-fine-tune.
- The augmented TABL models obtained by our method with two implementation strategies are denoted as aTABL-IS1 and aTABL-IS2, respectively. Similarly, the augmented CNN model denoted as aCNN. Here we should note that, similar to aTABL, there are also two implementation strategies for aCNN. In our experiments, we simply used the second implementation strategy for aCNN.
- In addition to the above, we also train a TABL model from scratch (random initialization) using both D_{old} and D_{new} . This model, denoted as TABL, represents the scenario where we have access to both the old and new datasets at once. Similarly, CNN is used to denote the model that was trained using both D_{old} and D_{new} .

To find the best network architecture for the pre-trained models (TABL-base), several configurations for hidden layers were validated. We followed the design in [29], i.e., all hidden layers except the last one are Bilinear layers and the prediction layer is a TABL. Grid search was used for hyperparameter tuning. The results of ablation study for hidden layers can be seen in B. The best network architectures for each pre-trained model corresponding to each experiment can be seen in Figure 6. For the CNN architecture, we adopted a conventional design pattern of 1D CNN for time-series, which is shown in Figure 2.

Tables 2 and 3 provide the prediction performance of all models on the test sets defined on each experiment. The results are grouped based on the target stock S . At the bottom of Tables 2 and 3, we report the average performance over the five target stocks. In addition, the last column shows the maximum rank value (chosen through validation) associated with our methods. As we have described two implementation strategies for the augmented TABL in Section

Table 2

Performance of TABL networks (Mean $\pm$ Standard Deviation) on the test sets of the Experimental Setup 1.

Target	Method	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	Max Rank
Stock 1	TABL	62.40 $\pm$ 00.10	62.10 $\pm$ 00.10	62.10 $\pm$ 00.10	62.08 $\pm$ 00.10	-
	TABL-base	63.96 $\pm$ 00.20	65.01 $\pm$ 00.30	64.45 $\pm$ 00.30	63.98 $\pm$ 00.20	-
	TABL-fine-tune	64.97 $\pm$ 00.40	66.66 $\pm$ 00.50	65.73 $\pm$ 00.40	64.94 $\pm$ 00.40	-
	aTABL-IS1	65.37 $\pm$ 00.30	68.07 $\pm$ 00.50	66.34 $\pm$ 00.30	65.28 $\pm$ 00.30	9
	aTABL-IS2	65.41 $\pm$ 00.40	68.68 $\pm$ 00.70	66.49 $\pm$ 00.40	65.28 $\pm$ 00.40	8
Stock 2	TABL	78.44 $\pm$ 00.10	74.64 $\pm$ 00.20	66.26 $\pm$ 00.00	69.43 $\pm$ 00.00	-
	TABL-base	79.48 $\pm$ 00.40	77.21 $\pm$ 00.012	66.49 $\pm$ 00.10	70.31 $\pm$ 00.20	-
	TABL-fine-tune	79.95 $\pm$ 01.00	79.97 $\pm$ 3.0	65.22 $\pm$ 00.10	69.91 $\pm$ 00.70	-
	aTABL-IS1	80.86 $\pm$ 00.30	82.62 $\pm$ 01.40	65.77 $\pm$ 00.20	70.98 $\pm$ 00.20	1
	aTABL-IS2	81.07 $\pm$ 00.30	83.48 $\pm$ 01.00	65.72 $\pm$ 00.10	71.11 $\pm$ 00.20	4
Stock 3	TABL	84.94 $\pm$ 00.10	75.65 $\pm$ 00.30	66.22 $\pm$ 00.10	70.03 $\pm$ 00.20	-
	TABL-base	82.96 $\pm$ 01.30	71.14 $\pm$ 02.50	66.17 $\pm$ 00.20	68.25 $\pm$ 01.20	-
	TABL-fine-tune	84.61 $\pm$ 01.30	76.35 $\pm$ 04.10	64.79 $\pm$ 00.50	69.04 $\pm$ 01.10	-
	aTABL-IS1	85.82 $\pm$ 00.40	80.01 $\pm$ 01.90	65.26 $\pm$ 00.10	70.58 $\pm$ 00.50	4
	aTABL-IS2	85.01 $\pm$ 01.90	77.74 $\pm$ 06.70	65.60 $\pm$ 00.30	70.02 $\pm$ 01.90	1
Stock 4	TABL	77.55 $\pm$ 00.10	71.76 $\pm$ 00.20	64.25 $\pm$ 00.20	66.41 $\pm$ 00.10	-
	TABL-base	76.27 $\pm$ 03.20	69.47 $\pm$ 05.30	66.98 $\pm$ 00.60	67.74 $\pm$ 02.40	-
	TABL-fine-tune	80.96 $\pm$ 00.40	80.66 $\pm$ 00.90	65.33 $\pm$ 00.80	70.20 $\pm$ 00.80	-
	aTABL-IS1	81.45 $\pm$ 00.30	81.10 $\pm$ 01.30	66.70 $\pm$ 00.30	71.48 $\pm$ 00.20	1
	aTABL-IS2	81.38 $\pm$ 00.50	80.87 $\pm$ 01.60	66.55 $\pm$ 00.30	71.32 $\pm$ 00.40	1
Stock 5	TABL	76.94 $\pm$ 00.10	70.03 $\pm$ 00.20	59.90 $\pm$ 00.20	63.39 $\pm$ 00.20	-
	TABL-base	67.72 $\pm$ 00.40	56.83 $\pm$ 00.70	59.82 $\pm$ 00.30	57.81 $\pm$ 00.10	-
	TABL-fine-tune	77.35 $\pm$ 00.90	68.61 $\pm$ 01.50	64.57 $\pm$ 00.70	66.16 $\pm$ 01.20	-
	aTABL-IS1	78.77 $\pm$ 02.10	72.24 $\pm$ 04.00	64.43 $\pm$ 01.40	67.18 $\pm$ 02.60	6
	aTABL-IS2	78.50 $\pm$ 02.90	71.53 $\pm$ 06.00	63.59 $\pm$ 03.20	66.47 $\pm$ 04.10	4
Average	TABL	76.05 $\pm$ 08.30	70.84 $\pm$ 05.40	63.75 $\pm$ 02.70	66.27 $\pm$ 03.50	-
	TABL-base	74.08 $\pm$ 08.00	67.93 $\pm$ 07.60	64.78 $\pm$ 02.90	65.62 $\pm$ 04.90	-
	TABL-fine-tune	77.57 $\pm$ 07.50	74.45 $\pm$ 06.50	65.13 $\pm$ 00.50	68.05 $\pm$ 02.40	-
	aTABL-IS1	78.45 $\pm$ 07.80	76.81 $\pm$ 06.30	65.70 $\pm$ 00.90	69.10 $\pm$ 02.70	-
	aTABL-IS2	78.27 $\pm$ 07.60	76.46 $\pm$ 06.20	65.59 $\pm$ 01.20	68.84 $\pm$ 02.80	-

generalization performance of the model. However, we can see that good performance can be obtained from low values of the rank.

To have a better understanding of the prediction performance on each stock, we provide in Figure 3 the confusion matrices obtained by using the best model on each stock from the models listed in Tables 2 and 3. As can be seen, for most of the stocks, the stationary class (label 0) is the most populated one. The results show that for all stocks the majority of the predictions are to the stationary class (label 0), while the models can distinguish between the stationary class and the two other classes (up and down corresponding to labels 1 and 2, respectively). Focusing on the classes up and down, we can see that when a time-series is classified to these two classes, it is correct in most of the cases, while misclassifications are mostly directed to the stationary class. When excluding the stationary class, i.e., considering only the cases when the models predict that the mid-price will move up or down which are the cases that would lead to an action, distinguishing between class up (label 1) and class down (label 2) is very accurate. Another performance measure based on the provided confusion matrix is the win-rate, which is calculated by assuming that trades would only be placed when the forecast indicates that the stock price will change. As the prediction of changes is more difficult and important than the prediction of stationary class, the win-rate shows the performance of the proposed method in predicting class 1 and class 2. Table 4 shows the win-rate of all stocks' confusion matrices. As can be seen, our method achieves high performance in predicting the changes.

Table 3Performance of CNNs (Mean $\pm$ Standard Deviation) on the test sets of the Experimental Setup 1.

Target	Method	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	Max Rank
Stock 1	CNN	53.93 $\pm$ 0.013	53.69 $\pm$ 0.014	53.05 $\pm$ 0.012	52.45 $\pm$ 0.013	-
	CNN-base	52.94 $\pm$ 0.005	52.4 $\pm$ 0.005	51.78 $\pm$ 0.005	50.78 $\pm$ 0.006	-
	CNN-fine-tune	55.07 $\pm$ 0.0	54.99 $\pm$ 0.005	54.67 $\pm$ 0.005	54.52 $\pm$ 0.006	-
	aCNN	54.74$\pm$0.001	55.6$\pm$0.002	54.92$\pm$0.001	54.77$\pm$0.001	2
Stock 2	CNN	77.24 $\pm$ 0.013	73.43 $\pm$ 0.033	64.93 $\pm$ 0.005	67.96 $\pm$ 0.009	-
	CNN-base	76.89 $\pm$ 0.003	72.19 $\pm$ 0.007	65.07 $\pm$ 0.002	67.76 $\pm$ 0.003	-
	CNN-fine-tune	78.71 $\pm$ 0.009	77.13 $\pm$ 0.024	64.73 $\pm$ 0.002	68.86 $\pm$ 0.007	-
	aCNN	78.48$\pm$0.008	75.73$\pm$0.022	65.37$\pm$0.001	69.01$\pm$0.006	1
Stock 3	CNN	82.93 $\pm$ 0.013	71.77 $\pm$ 0.034	64.92 $\pm$ 0.006	67.67 $\pm$ 0.01	-
	CNN-base	83.03$\pm$0.01	71.64$\pm$0.025	64.99$\pm$0.002	67.77$\pm$0.009	-
	CNN-fine-tune	76.45 $\pm$ 0.042	64.18 $\pm$ 0.029	61.4 $\pm$ 0.034	60.97 $\pm$ 0.042	-
	aCNN	82.64 $\pm$ 0.008	70.49 $\pm$ 0.021	65.32 $\pm$ 0.003	67.55 $\pm$ 0.007	1
Stock 4	CNN	76.9 $\pm$ 0.022	70.95 $\pm$ 0.041	66.74 $\pm$ 0.005	68.16 $\pm$ 0.013	-
	CNN-base	74.3 $\pm$ 0.028	66.6 $\pm$ 0.035	66.16 $\pm$ 0.017	66.06 $\pm$ 0.026	-
	CNN-fine-tune	79.11 $\pm$ 0.028	79.98 $\pm$ 0.057	63.44 $\pm$ 0.034	67.86 $\pm$ 0.037	-
	aCNN	79.42$\pm$0.0	74.93$\pm$0.002	67.17$\pm$0.0	70.13$\pm$0.001	2
Stock 5	CNN	67.34 $\pm$ 0.032	56.72 $\pm$ 0.014	56.75 $\pm$ 0.019	56.19 $\pm$ 0.003	-
	CNN-base	67.89 $\pm$ 0.024	56.29 $\pm$ 0.022	56.24 $\pm$ 0.012	56.06 $\pm$ 0.016	-
	CNN-fine-tune	67.52 $\pm$ 0.055	60.78 $\pm$ 0.034	53.0 $\pm$ 0.021	52.69 $\pm$ 0.025	-
	aCNN	67.99$\pm$0.006	56.49$\pm$0.006	57.11$\pm$0.002	56.69$\pm$0.002	2
Average	CNN	72.15 $\pm$ 0.11	66.27 $\pm$ 0.089	61.78 $\pm$ 0.057	63.19 $\pm$ 0.071	-
	CNN-base	71.82 $\pm$ 0.105	64.56 $\pm$ 0.085	61.23 $\pm$ 0.058	62.24 $\pm$ 0.072	-
	CNN-fine-tune	72.63 $\pm$ 0.089	68.56 $\pm$ 0.102	59.59 $\pm$ 0.054	61.3 $\pm$ 0.074	-
	aCNN	72.77$\pm$0.101	66.54$\pm$0.092	62.0$\pm$0.051	63.6$\pm$0.068	-

Table 4

Win-rate measured on the test set of each stock.

	Stock 1	Stock 2	Stock 3	Stock 4	Stock 5
Win-Rate(%)	74.6	85.5	79.8	82.3	64.6

Table 5Performance of TABL architectures (Mean $\pm$ Standard Deviation) measured on the test set in the Experiment Setup 2.

Method	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	#Params
TABL	77.54 $\pm$ 01.20	69.26 $\pm$ 03.30	65.80 $\pm$ 01.10	67.33 $\pm$ 02.10	17,914
TABL-fine-tune	79.17 $\pm$ 01.30	72.56 $\pm$ 01.10	66.17 $\pm$ 03.10	68.77 $\pm$ 02.30	53,742
aTABL-IS1	80.31 $\pm$ 03.10	73.13 $\pm$ 02.40	66.26 $\pm$ 02.10	69.73 $\pm$ 03.00	26,476
aTABL-IS2	80.56 $\pm$ 02.30	75.80 $\pm$ 01.10	66.47 $\pm$ 03.10	70.00 $\pm$ 03.20	25,636

4.3. Experimental Setup 2

To have a better real-world understanding of the advantages of the proposed method, we define another experimental setup as follows: the old dataset $\mathcal{D}_{old}$ consists of three stocks from the FI-2010 database and the new dataset $\mathcal{D}_{new}$ contains the remaining two stocks. We only have access to a pre-trained model $\mathcal{N}_{old}$ that has been trained on $\mathbf{T}_{old}$ but not to its training data $\mathcal{D}_{old}$. The objective is to build a model or a set of models that work well not only for the stocks in $\mathcal{D}_{new}$ but also for the stocks in $\mathcal{D}_{old}$. There are three approaches to tackle this problem:

- We simply keep the pre-trained model TABL-base or CNN-base and use it for all five stocks. The results related to this approach are denoted with TABL-base and CNN-base, respectively.



Figure 3: Confusion matrices for the best model on the test set of each stock among the models in Tables 2 and 3. Class 0 refers to the stock price remains the same, class 1 to the situation where the price goes up, and class 2 is where the price goes downs.

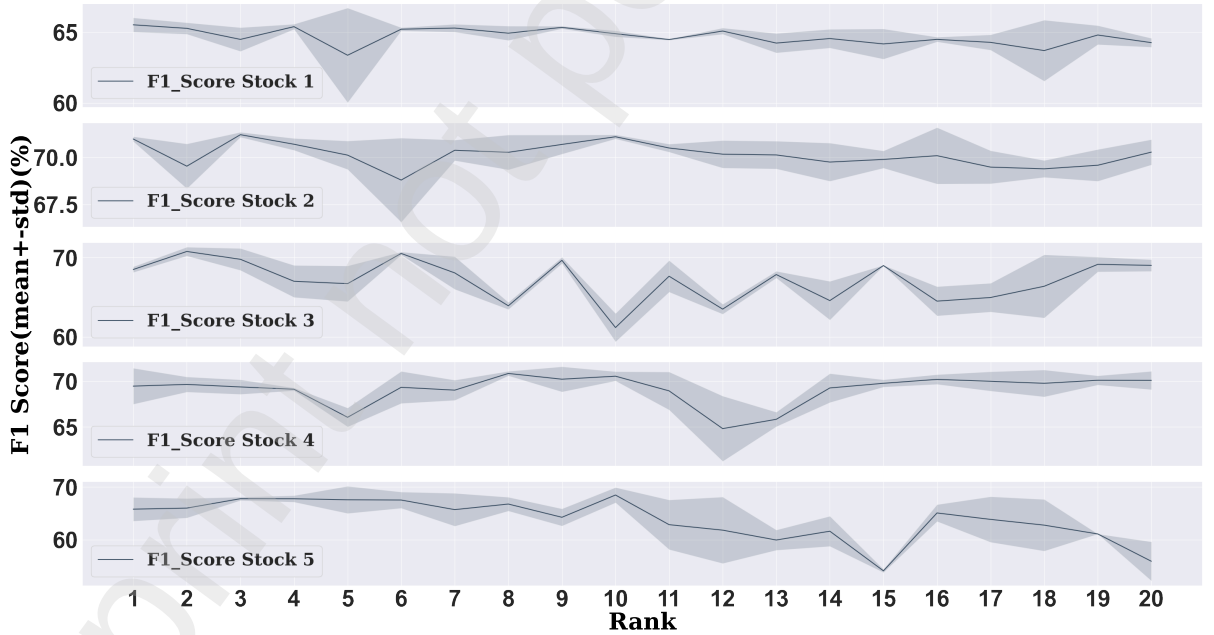


Figure 4: Performance of aTABL-IS1 for values of K (rank of the model's weight matrices) between 1 to 20. The shadow of each line shows the standard deviation of results for each rank.

- For each new stock in D_{new} , we make a copy of the TABL-base or CNN-base model and finetune it using the data coming from the new stock. The fine-tuned model is used to generate predictions for the new stock. This approach requires the storage of three models: the pre-trained model trained on D_{old} and two models fine-tuned on data of

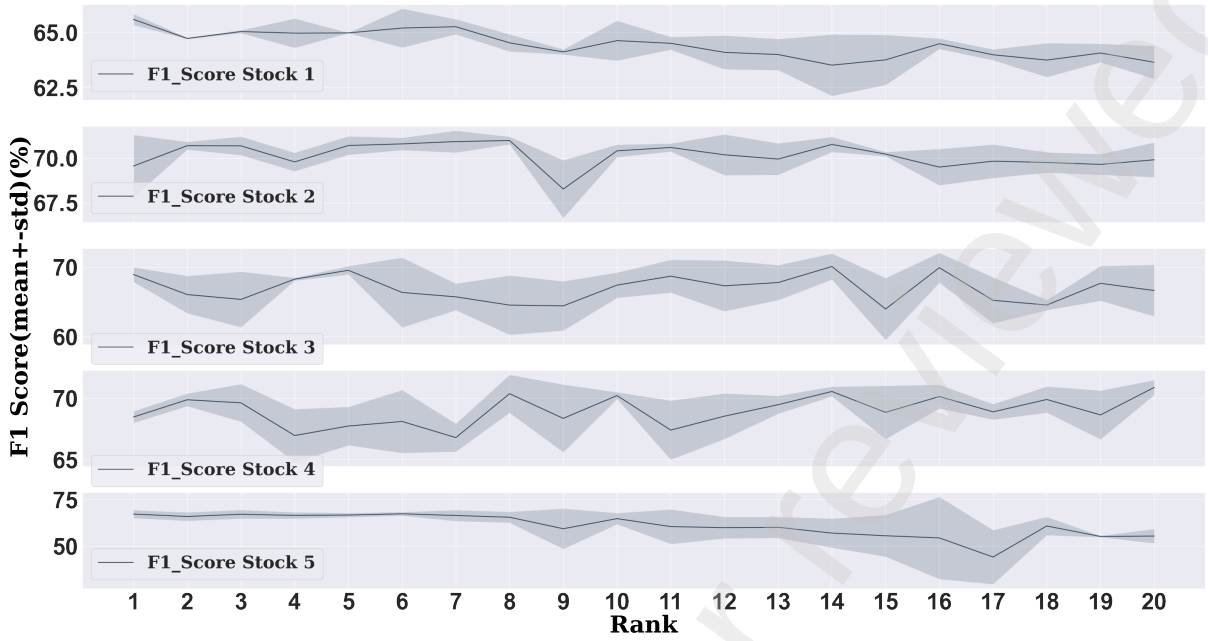


Figure 5: Performance of aTABL-IS2 for values of K (rank of the model's weight matrices) between 1 to 20. The shadow of each line shows the standard deviation of results for each rank.

Table 6

Performance of CNN architectures (Mean $\pm$ Standard Deviation) measured on the test set in the Experiment Setup 2.

Method	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	#Params
CNN	72.33 $\pm$ 0.025	62.83 $\pm$ 0.024	62.43 $\pm$ 0.007	62.36 $\pm$ 0.015	17,091
CNN-fine-tune	73.89 $\pm$ 0.007	64.18 $\pm$ 0.01	63.71 $\pm$ 0.003	63.89 $\pm$ 0.005	51,273
aCNN	73.27 $\pm$ 0.01	63.62 $\pm$ 0.014	62.56 $\pm$ 0.005	62.94 $\pm$ 0.006	30,427

the two new stocks $D_{\text{new},4}$ and $D_{\text{new},5}$. The results related to this approach are denoted with TABL-fine-tune and CNN-fine-tune.

- Using the proposed method, in which for each of the new stocks an augmented model is created by adapting the pre-trained model using the corresponding datasets $D_{\text{new},4}$ and $D_{\text{new},5}$, respectively. Using our approach, we can store the pre-trained model $\mathcal{N}_{\text{old}}$ and only the auxiliary parameters for augmented models trained on $D_{\text{new},4}$ and $D_{\text{new},5}$. The results related to this approach are denoted with aTABL-IS1 and aTABL-IS2, corresponding to two implementations of augmented TABL, and the results for augmented CNNs are denoted with aCNN.

With this experimental setup, we compare not only the prediction performance of three different approaches but also the storage cost associated with them. Tables 5 and 6 show the prediction performance as well as the total number of parameters that need to be stored for each case. From Table 5, we can easily observe that the proposed method not only outperforms the finetuning approach in terms of performance but also in storage cost. In practice, when we have N new stocks in our portfolio, the finetuning approach would require additional storage of N models. On the other hand, for every new stock, our approach only requires a small fraction of additional storage for the auxiliary parameters. Even though the proposed augmentation has slightly inferior performance compared to the standard finetuning approach using CNNs, we still observe performance gains compared to the baseline CNN in Table 6. Regarding the storage cost, the proposed method always leads to storage savings compared to the finetuning approach.

4.4. Online Learning Experimental Setup

Since the proposed model augmentation approach can also be used in an online learning setting in which new data of the same stock is generated through time, we also conducted experiments simulating this setting. More specifically,

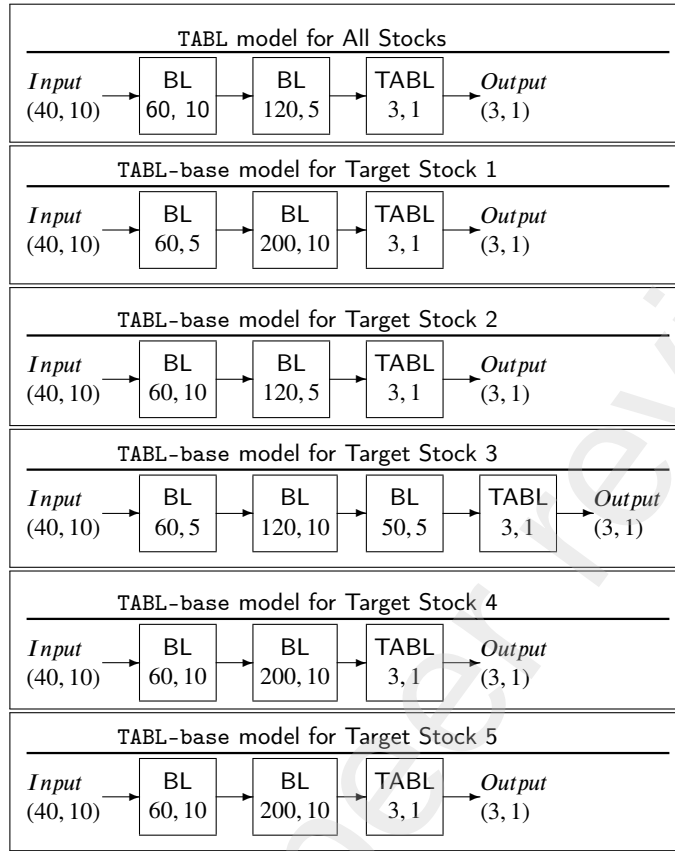


Figure 6: The best base model network topology for each target stock. Each block corresponds to a layer (BL or TABL) and indicates the the output dimensions of the layer. The output with shape of (3,1) is a column vector with the 3 probability-like outputs of the network corresponding to the three classes. The top topology (TABL) refers to the case training is conducted using the training sets of all stocks.

Table 7

Performance of CNN architectures (Mean $\pm$ Standard Deviation) measured on the test set in the online learning experimental setup

Method	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	Max rank
CNN-base	74.05 $\pm$ 0.016	64.56 $\pm$ 0.021	63.9 $\pm$ 0.008	64.13 $\pm$ 0.014	-
CNN-fine-tune	74.94 $\pm$ 0.005	65.8 $\pm$ 0.007	63.79 $\pm$ 0.003	64.69 $\pm$ 0.002	-
aCNN	75.24$\pm$0.013	66.26$\pm$0.02	63.94$\pm$0.003	64.93$\pm$0.007	1

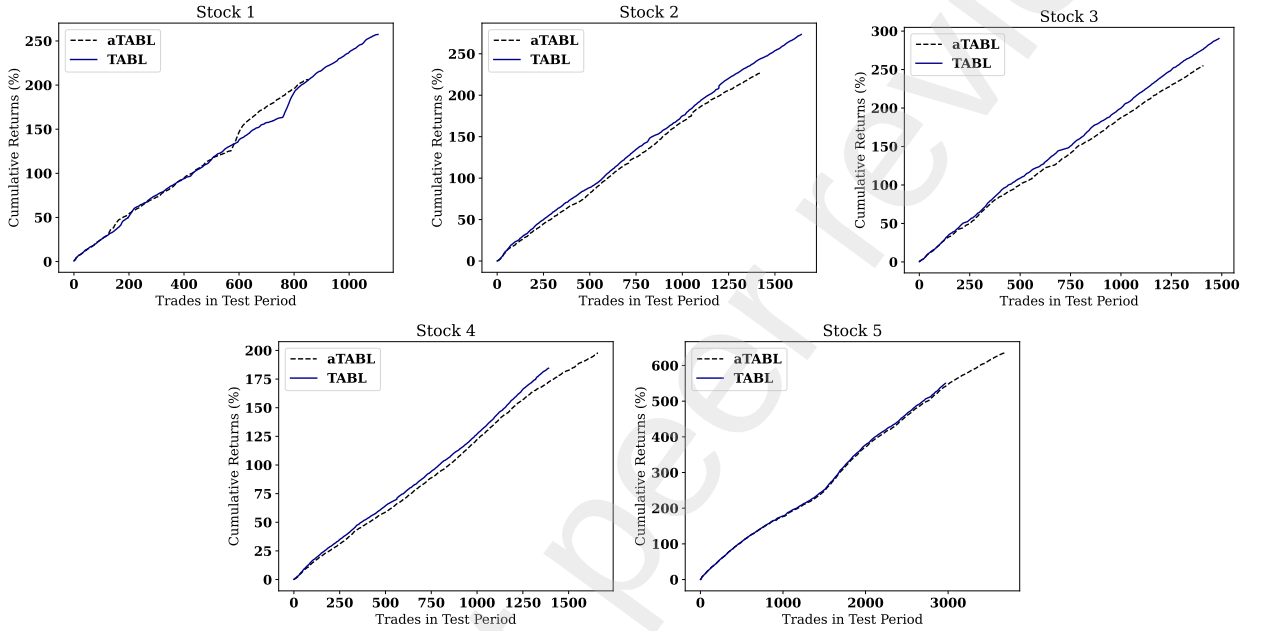
data from the first five days were used to train the base models. In order to make predictions for the eighth, ninth, and tenth days, there are three approaches:

- We simply use the base models (TABL-base and CNN-base).
- We fine-tune the base models using data from the sixth and seventh days before making predictions for the last three days (results denoted with TABL-fine-tune and CNN-fine-tune).
- use our model augmentation method with the data from the sixth and seventh days to adapt the baseline models (results denoted with aTABL-IS1, aTABL-IS2 and aCNN).

Table 8

Performance of TABL architectures (Mean $\pm$ Standard Deviation) measured on the test set in the online learning experimental setup

Method	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	Max rank
TABL-base	75.15 $\pm$ 0.02	66.61 $\pm$ 0.024	65.43 $\pm$ 0.003	65.58 $\pm$ 0.015	-
TABL-fine-tune	76.79 $\pm$ 0.001	68.33 $\pm$ 0.002	66.12 $\pm$ 0.001	67.13 $\pm$ 0.001	-
aTABL-IS1	79.88$\pm$0.02	76.16$\pm$0.045	64.93$\pm$0.006	68.68$\pm$0.018	9
aTABL-IS2	80.99$\pm$0.006	78.15$\pm$0.024	65.61$\pm$0.005	69.92$\pm$0.004	8

**Figure 7:** Cumulative Returns (%) of Trades in Test Period.

Results obtained by following these three approaches are shown in Tables 7 and 8 and indicate that the proposed model augmentation method is also effective in adapting a pre-trained model in an online learning manner, yielding better performance compared to the baseline models as well as the fine-tuned models.

Here we should note that, in general, deep learning models may achieve different performances when trained using different random parameter initializations and when tested on data with different characteristics. We can see that the standard deviations of the performance of the models in this experiment, as well as in all other experiments reported above, are small in value. This shows that the adopted models provide reliable performance in terms of the stochastic nature of the Backpropagation algorithm.

4.5. Trading Simulation

An efficient trading system requires complex trading decisions and strategies. Forecasting price movements is an important part of a trading system. However, a more important component in a profitable system is the order placing strategy, i.e., when to place an order, at what price and volume to place the order and so on. The design of an efficient order placing strategy is out of the scope of this paper. This paper focuses on improving the prediction performance of mid-price direction movements, which can help develop more accurate trading systems.

To evaluate the profitability of the proposed model, we defined a simple long-only trading system with a naive order placing rule as follows: when the prediction is “up” we will buy one share at best ask price and hold until the predicted label changes to “down” (we do nothing for stationary predictions). At this point, we will sell at the best bid price to take the transaction costs on bid-ask spread into account (the pricing scheme of a particular broker, the

trading tier, and the cost of data subscription are not included). To calculate the actual cumulative return, we use the un-normalized ask and bid price values which are not included in the public FI-2010 dataset [61]. We do this because the normalization of the public data at the element-level leads to prices which can have negative values. The plots in the Figure 7 show the cumulative returns of trades for the test dataset of each stock for the TABL and the best aTABL models in Table 2. The return of each trade is calculated as follows:

$$Returns = (Exit_{Price} / Entry_{Price}) - 1, \quad (26)$$

where the $Exit_{Price}$ is the best bid price and $Entry_{Price}$ is the best ask price of the above trading strategy. The results in Figure 7 show that trading based on the predictions of TABL and aTABL is profitable. The plots of stocks 4 and 5 show that the predictions of aTABL lead to better cumulative returns. The demonstrated results are based on a naive trading strategy. In reality, there are many other factors that affect the actual profit of a trading system. These include the cost of research and development, the cost of running and maintaining the trading system, the additional transaction cost on top of the bid-ask spread, market impact of transactions, and the scalability of the trade are very important factors in a live trading system. A practical trading system is designed, developed and evaluated using a complex procedure considering many different information besides the forecasting ability of a model. This trading simulation is designed to show the potential of the proposed method in developing a trading system by relaxing other factors that affect trading.

5. Conclusion

In this paper, we studied a new research problem defined on financial time-series data, that of efficiently training new models for stock mid-price time-series classification by exploiting knowledge in existing deep learning models trained on time-series data of different stocks or time-series data of past periods. We proposed a new method that exploits model augmentation and low-rank matrix approximation to improve the prediction performance and reduce the storage cost. Our model augmentation approach takes advantage of the learned information from the knowledge encoded in the parameters of an existing (pre-trained) model and learns auxiliary connections that are added to the pre-trained model to adapt it for the new task. The low rank approximation of auxiliary parameters regularizes the learning process and reduces the storage cost of new models. Extensive experiments on stock mid-price direction prediction tasks demonstrated that the proposed method can lead to performance improvements as well as a reduction in storage requirements during deployment. Interesting future research directions include investigating the effectiveness of the proposed approach in time-series classification problems coming from different applications, as well as the study of designing deep learning models targeting applications involving other forms of input data, like images, videos, audio, under the restrictions indicated by the adopted problem formulation.

Acknowledgment

The research received funding from the Independent Research Fund Denmark project DISPA (Project Number: 9041-00004).

A. Appendix

Let us denote by N the number of samples in a mini-batch during stochastic optimization. Below, we provide the computational complexity estimate for two implementation strategies described in Section 3.

Computational Complexity of Implementation Strategy 1

- Computing Eq. (16) requires $NDD'T$ operations.
- Computing Eq. (17) requires $ND'TT$ operations.
- Computing Eq. (18) requires $ND'TT' + 2ND'T'$ operations.

Computational Complexity of Implementation Strategy 2

- Computing Eq. (19) requires $NDD'T + NDKT + NKT D'$ operations.

Table 9Ablation study of $\mathcal{N}_{old}$ for Stock 1

Model	F1-score
layers_ [[60, 5], [200, 10]]	0.694022
layers_ [[60, 10], [120, 5]]	0.687618
layers_ [[60, 5], [120, 10], [50, 5]]	0.686768
layers_ [[60, 10], [200, 10]]	0.68425
layers_ [[50, 10], [250, 5]]	0.68172
layers_ [[60, 10], [200, 5]]	0.67418

Table 10Ablation study of $\mathcal{N}_{old}$ for Stock 2

Model	F1-score
layers_ [[60, 10], [120, 5]]	0.701169
layers_ [[60, 10], [200, 5]]	0.693212
layers_ [[60, 5], [120, 10], [50, 5]]	0.692069
layers_ [[60, 5], [200, 10]]	0.685344
layers_ [[60, 10], [200, 10]]	0.68498
layers_ [[50, 10], [250, 5]]	0.683752

Table 11Ablation study of $\mathcal{N}_{old}$ for Stock 3

Model	F1-score
layers_ [[60, 5], [120, 10], [50, 5]]	0.701476
layers_ [[60, 10], [120, 5]]	0.699135
layers_ [[60, 5], [200, 10]]	0.698294
layers_ [[60, 10], [200, 5]]	0.698241
layers_ [[50, 10], [250, 5]]	0.698197
layers_ [[60, 10], [200, 10]]	0.689825

- Computing Eq. (20) requires $ND'TT + ND'TK + ND'KT$ operations.
- Computing Eq. (21) requires $ND'TT' + ND'TK + ND'KT' + 2ND'T'$ operations.

B. Appendix

To find the best topology for $\mathcal{N}_{old}$ for each target stock we ran experiments for different topologies and hyperparameter values. Tables 9 - 13 show the results of the topology ablation study for $\mathcal{N}_{old}$ of each target stock.

Table 12Ablation study of $\mathcal{N}_{old}$ for Stock 4

Model	F1-score
layers_ [[60, 10], [200, 10]]	0.698838
layers_ [[60, 5], [120, 10], [50, 5]]	0.69745
layers_ [[60, 10], [120, 5]]	0.697304
layers_ [[60, 5], [200, 10]]	0.691858
layers_ [[50, 10], [250, 5]]	0.687543
layers_ [[60, 10], [200, 5]]	0.685277

Table 13Ablation study of $\mathcal{N}_{old}$ for Stock 5

Model	F1-score
layers_ [[60, 10], [200, 10]]	0.710281
layers_ [[60, 5], [120, 10], [50, 5]]	0.709831
layers_ [[60, 10], [120, 5]]	0.706329
layers_ [[50, 10], [250, 5]]	0.706135
layers_ [[60, 10], [200, 5]]	0.70475
layers_ [[60, 5], [200, 10]]	0.70228

References

- [1] Avraam Tsantekidis, Nikolaos Passalis, Anastasios Tefas, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis. Forecasting stock prices from the limit order book using convolutional neural networks. In *IEEE Conference on Business Informatics*, volume 1, pages 7–12, 2017.
- [2] Zihao Zhang, Stefan Zohren, and Stephen Roberts. Deeplob: Deep convolutional neural networks for limit order books. *IEEE Transactions on Signal Processing*, 67:3001–3012, 2019.
- [3] Nikolaos Passalis, Anastasios Tefas, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis. Temporal logistic neural bag-of-features for financial time series forecasting leveraging limit order book data. *Pattern Recognition Letters*, 136:183–189, 2020.
- [4] N. Passalis, A. Tefas, J. Kannianen, M. Gabbouj, and A. Iosifidis. Deep adaptive input normalization for time series forecasting. *IEEE Transactions on Neural Networks and Learning Systems*, 31(9):3760–3765, 2020.
- [5] Justin A. Sirignano. Deep learning for limit order books. *Quantitative Finance*, 19(4):549–570, 2019.
- [6] Matthew Francis Dixon. Sequence classification of the limit order book using recurrent neural networks. *SSRN Electronic Journal*, 2017.
- [7] Dat Thanh Tran, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis. Data-driven neural architecture learning for financial time-series forecasting. *ArXiv*, abs/1903.06751, 2019.
- [8] Defu Cao, Yousef El-Laham, Loc Trinh, Svitlana Vyetenko, and Yan Liu. DSLOB: A synthetic limit order book dataset for benchmarking forecasting algorithms under distributional shift. *CoRR*, abs/2211.11513, 2022. doi: 10.48550/arXiv.2211.11513.
- [9] Artemios-Anargyros Semenoglou, Evangelos Spiliotis, and Vassilios Assimakopoulos. Data augmentation for univariate time series forecasting with neural networks. *Pattern Recognition*, 134:109132, 2023. ISSN 0031-3203.
- [10] Omer Berat Sezer, Mehmet Ugur Gudelek, and Ahmet Murat Ozbayoglu. Financial time series forecasting with deep learning: A systematic literature review: 2005–2019. *Applied Soft Computing*, 90:106181, 2020.
- [11] Serkan Kiranyaz, Turker Ince, Alexandros Iosifidis, and Moncef Gabbouj. Progressive operational perceptrons. *Neurocomputing*, 224:142–154, 2017.
- [12] Dat Thanh Tran and Alexandros Iosifidis. Learning to rank: A progressive neural network learning approach. In *IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 8355–8359, 2019.
- [13] Dat Thanh Tran, Serkan Kiranyaz, Moncef Gabbouj, and Alexandros Iosifidis. Progressive operational perceptrons with memory. *Neurocomputing*, 379:172–181, 2020.
- [14] Serkan Kiranyaz, Turker Ince, Alexandros Iosifidis, and Moncef Gabbouj. Operational neural networks. *Neural Computing and Applications*, pages 1–24, 2020.
- [15] Max Jaderberg, Andrea Vedaldi, and Andrew Zisserman. Speeding up convolutional neural networks with low rank expansions. *arXiv preprint arXiv:1405.3866*, 2014.
- [16] Dat Thanh Tran, Alexandros Iosifidis, and Moncef Gabbouj. Improving efficiency in convolutional neural networks with multilinear filters. *Neural Networks*, 105:328–339, 2018.

- [17] Hantao Huang and Hao Yu. Ltnn: A layerwise tensorized compression of multilayer neural network. *IEEE transactions on neural networks and learning systems*, 30(5):1497–1511, 2018.
- [18] Xiaofeng Ruan, Yufan Liu, Chunfeng Yuan, Bing Li, Weiming Hu, Yangxi Li, and Stephen Maybank. Edp: An efficient decomposition and pruning scheme for convolutional neural network compression. *IEEE Transactions on Neural Networks and Learning Systems*, 32(10):4499–4513, 2020.
- [19] Ling Shao, Fan Zhu, and Xuelong Li. Transfer learning for visual categorization: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 26(5):1019–1034, 2014.
- [20] Zhengming Ding and Yun Fu. Deep transfer low-rank coding for cross-domain learning. *IEEE Transactions on Neural Networks and Learning Systems*, 30(6):1768–1779, 2018.
- [21] Elizabeth Fons, Paula Dawson, Xiao-jun Zeng, John Keane, and Alexandros Iosifidis. Augmenting transferred representations for stock classification. *arXiv preprint arXiv:2011.04545*, 2020.
- [22] Lixin Duan, Dong Xu, and Ivor Wai-Hung Tsang. Domain adaptation from multiple sources: A domain-dependent regularization approach. *IEEE Transactions on Neural Networks and Learning Systems*, 23(3):504–518, 2012.
- [23] Zengmao Wang, Bo Du, and Yuhong Guo. Domain adaptation with neural embedding matching. *IEEE Transactions on Neural Networks and Learning Systems*, 31(7):2387–2397, 2019.
- [24] L. Hedegaard, O.A. Sheikh-Omar, and A. Iosifidis. Supervised Domain Adaptation; a Graph Embedding perspective and a rectified experimental protocol. *IEEE Transactions on Image Processing*, 30:8619–8631, 2021.
- [25] Sheng Wu, Ancong Wu, and Wei-Shi Zheng. Online deep transferable dictionary learning. *Pattern Recognition*, 118:108007, 2021. ISSN 0031-3203.
- [26] Lean Yu, Shouyang Wang, and Kin Keung Lai. An online learning algorithm with adaptive forgetting factors for feedforward neural networks in financial time series forecasting. *Nonlinear dynamics and systems theory*, 7(1):51–66, 2007.
- [27] Rodolfo C Cavalcante and Adriano LI Oliveira. An approach to handle concept drift in financial time series based on extreme learning machines and explicit drift detection. In *international joint conference on neural networks*, pages 1–8, 2015.
- [28] Xinying Wang and Min Han. Online sequential extreme learning machine with kernels for nonstationary time series prediction. *Neurocomputing*, 145:90–97, 2014.
- [29] Dat Thanh Tran, Alexandros Iosifidis, Juho Kannianen, and Moncef Gabbouj. Temporal attention-augmented bilinear network for financial time-series data analysis. *IEEE Transactions on Neural Networks and Learning Systems*, 30(5):1407–1418, 2019.
- [30] Kabin Kanjamapornkul, Richard Pinčák, Sanphet Chunitipaisan, and Erik Bartoš. Support spinor machine. *Digital Signal Processing*, 70:59–72, 2017.
- [31] Kabin Kanjamapornkul, Richard Pinčák, and Erik Bartoš. The study of thai stock market across the 2008 financial crisis. *Physica A: Statistical Mechanics and its Applications*, 462:117–133, 2016.
- [32] Avraam Tsantekidis, Nikolaos Passalis, Anastasios Tefas, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis. Using deep learning for price prediction by exploiting stationary limit order book features. *Applied Soft Computing*, 93, 106401, 2020.
- [33] D. T. Tran, J. Kannianen, M. Gabbouj, and A. Iosifidis. Bilinear input normalization for neural networks in financial forecasting. *arXiv:2109.00983*, 2021.
- [34] Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2009.
- [35] Michael T Rosenstein, Zvika Marx, Leslie Pack Kaelbling, and Thomas G Dietterich. To transfer or not to transfer. In *NIPS 2005 Workshop on Transfer Learning*, volume 898, pages 1–4, 2005.
- [36] Hassan Ismail Fawaz, Germain Forestier, Jonathan Weber, Lhassane Idoumghar, and Pierre-Alain Muller. Transfer learning for time series classification. In *IEEE International Conference on Big Data*, pages 1367–1376, 2018.
- [37] Donald J Berndt and James Clifford. Using dynamic time warping to find patterns in time series. In *KDD workshop*, volume 10, pages 359–370, 1994.
- [38] Mahardhika Pratama, Marcus de Carvalho, Renchunzi Xie, Edwin Lughofer, and Jie Lu. Atl: Autonomous knowledge transfer from many streaming processes. In *ACM International Conference on Information and Knowledge Management*, pages 269–278, 2019.
- [39] João Gama, Indrè Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37, 2014.
- [40] Rui Ye and Qun Dai. A novel transfer learning framework for time series forecasting. *Knowledge-Based Systems*, 156:74–99, 2018.
- [41] Hua Zuo, Guangquan Zhang, Witold Pedrycz, Vahid Behbood, and Jie Lu. Fuzzy regression transfer learning in takagi–sugeno fuzzy models. *IEEE Transactions on Fuzzy Systems*, 25(6):1795–1807, 2016.
- [42] Thi-Thu Nguyen and Seokhoon Yoon. A novel approach to short-term stock price movement prediction using transfer learning. *Applied Sciences*, 9(22):4745, 2019.
- [43] Adriano Koshiyama, Sebastian Flennerhag, Stefano B Blumberg, Nick Firoozye, and Philip Treleaven. Quantnet: Transferring learning across systematic trading strategies. *arXiv preprint arXiv:2004.03445*, 2020.
- [44] Peilin Zhao, Steven CH Hoi, Jialei Wang, and Bin Li. Online transfer learning. *Artificial Intelligence*, 216:76–102, 2014.
- [45] Liang Ge, Jing Gao, and Aidong Zhang. Oms-tl: A framework of online multiple source transfer learning. In *ACM International Conference on Information & Knowledge Management*, pages 2423–2428, 2013.
- [46] David A Ross, Jongwoo Lim, Ruei-Sung Lin, and Ming-Hsuan Yang. Incremental learning for robust visual tracking. *International Journal of Computer Vision*, 77(1):125–141, 2008.
- [47] Mahardhika Pratama, Choiru Za'in, Andri Ashfahani, Yew Soon Ong, and Weiping Ding. Automatic construction of multi-layer perceptron network from streaming examples. In *ACM International Conference on Information and Knowledge Management*, pages 1171–1180, 2019.
- [48] Mei Wang and Weihong Deng. Deep visual domain adaptation: A survey. *Neurocomputing*, 312:135–153, 2018.

- [49] Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario Marchand, and Victor Lempitsky. Domain-adversarial training of neural networks. *The Journal of Machine Learning Research*, 17(1):2096–2030, 2016.
- [50] Mingsheng Long, Yue Cao, Jianmin Wang, and Michael Jordan. Learning transferable features with deep adaptation networks. In *International Conference on Machine Learning*, pages 97–105, 2015.
- [51] Arthur Gretton, Karsten M Borgwardt, Malte J Rasch, Bernhard Schölkopf, and Alexander Smola. A kernel two-sample test. *The Journal of Machine Learning Research*, 13(1):723–773, 2012.
- [52] Mahardhika Pratama, Marcus de Carvalho, Renchunzi Xie, Edwin Lughofer, and Jie Lu. Atl: Autonomous knowledge transfer from many streaming processes. *ACM International Conference on Information and Knowledge Management*, pages 269–278, 2019.
- [53] Xie Renchunzi and Mahardhika Pratama. Automatic online multi-source domain adaptation. *Information Sciences*, 582:480–494, 2022.
- [54] Adrian Bulat, Jean Kossaifi, Georgios Tzimiropoulos, and Maja Pantic. Incremental multi-domain learning with network latent tensor factorization. In *AAAI*, pages 10470–10477, 2020.
- [55] Rich Caruana. Multitask learning. *Machine learning*, 28(1):41–75, 1997.
- [56] Ali Senhaji, Jenni Raitoharju, Moncef Gabbouj, and Alexandros Iosifidis. Not all domains are equally complex: Adaptive multi-domain learning. *International Conference on Pattern Recognition*, 2020.
- [57] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
- [58] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 4700–4708, 2017.
- [59] Tamara G Kolda and Brett W Bader. Tensor decompositions and applications. *SIAM review*, 51(3):455–500, 2009.
- [60] Rama Cont. Statistical modeling of high-frequency financial data. *IEEE Signal Processing Magazine*, 28:16–25, 2011.
- [61] Adamantios Ntakaris, Martin Magris, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis. Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods. *Journal of Forecasting*, 37:852–866, 2018.
- [62] Dat Thanh Tran, Juho Kannianen, and Alexandros Iosifidis. How informative is the order book beyond the best levels? machine learning perspective. *NeurIPS 2021 Workshop on Machine Learning meets Econometrics*, 2021.